



Name: Shayan Umar

Designation: AI intern

Department: Artificial Intelligence

Date: 15-8-2025

Table of Contents

1. Theoretical Understanding:	3
1.1 Problem Framing:	3
1.2 Managing ML Projects:	8
1.3 Decision Forests:	18
1.4 Recommendation Systems:	22
1.5 Long-Short-Term-Memory (LSTMs):	26
1.6 Gated Recurrent Units (GRUs):	30
1.7 Encoder Decoder Architecture:	33
2. Project Work:	38
2.1 Text Generation (Character Level Prediction):	38
2.2 Capstone Project – Next Word Prediction:	40
2.3 Translation Task using encoder-decoder architecture:	41
3. Challenges and Approaches:	43
4. Result and Analysis:	44
5. Conclusion:	46
6. References:	47

1. Theoretical Understanding:

1.1 Problem Framing:

Problem framing is the process of analyzing a problem to isolate the individual elements that need to be addressed to solve it. Problem framing helps determine your project's technical feasibility and provides a clear set of goals and success criteria. When considering an ML solution, effective problem framing can determine whether or not your product ultimately succeeds.

Formal problem framing is the critical beginning for solving an ML problem, as it forces us to better understand both the problem and the data in order to design and build a bridge between them.

At a high level, ML problem framing consists of two distinct steps:

1. Determining whether ML is the right approach for solving a problem.
2. Framing the problem in ML terms.

1.1.1 Understand the problem:

Understand the problem, perform the following tasks:

- State the goal for the product you are developing or refactoring.
- Determine whether the goal is best solved using predictive ML, generative AI, or a non-ML solution.
- Verify you have the data required to train a model if you're using a predictive ML approach.

State the goal

- Begin by stating your goal in non-ML terms. The goal is the answer to the question, "What am I trying to accomplish?"

Clear use case for ML

Some view ML as a universal tool that can be applied to all problems. In reality, ML is a specialized tool suitable only for particular problems. You don't want to implement a complex ML solution when a simpler non-ML solution will work.

ML systems can be divided into two broad categories: predictive ML and generative AI. The following table lists their defining characteristics:

	Input	Output	Training technique
Predictive ML	Text Image Audio Video Numerical	Makes a prediction, for example, classifying an email as spam or not spam, guessing tomorrow's rainfall, or predicting the price of a stock. The output can typically be verified against reality.	Typically uses lots of data to train a supervised, unsupervised, or reinforcement learning model to perform a specific task.
Generative AI	Text Image Audio Video Numerical	Generates output based on the user's intent, for example, summarizing an article, or producing an audio clip or short video.	Typically uses lots of unlabeled data to train a large language model or image generator to fill in missing data. The model can then be used for tasks that can be framed as fill-in-the-blank tasks, or it can be fine-tuned by training it on labeled data for some specific task, like classification.

Before choosing ML, ensure your non-ML solution is fully optimized or try a heuristic-based manual approach. Use the non-ML solution as a benchmark to evaluate if ML is justified by considering:

1. **Quality** – Will ML provide a significant improvement, or is the current solution already near optimal?

2. **Cost & Maintenance** –

- Can ML justify higher compute/resource costs? (Even small gains in large systems may be worthwhile.)

- Does your team have the capacity for long-term ML maintenance and expertise?

Key Takeaway: ML is only worthwhile if it offers substantial benefits over a simpler, well-optimized baseline, and your team can sustain its costs and upkeep.

Key Requirements for Effective ML Data

1. Data Characteristics for Good Predictions:

- **Abundant:** More high-quality, relevant examples improve model performance.
- **Consistent & Reliable:** Data should be collected uniformly (e.g., same instruments, methods).
- **Trusted:** Prefer data from controlled, reliable sources (e.g., product logs) over uncertain ones (e.g., another ML system's output).
- **Available:** Features must be obtainable at prediction time; exclude hard-to-get features.
- **Correct:** Too many incorrect labels degrade model performance.
- **Representative:** Data must reflect real-world conditions to avoid poor generalization.

2. Predictive Power of Features:

- Features should correlate strongly with the target (e.g., temperature predicts rain better than day_of_week).
- **Methods to assess predictive power:**
 - Manual: Add/remove features during training to observe impact.
 - Automated: Use statistical methods like:
 - **Pearson correlation** (linear relationships)
 - **Adjusted Mutual Information (AMI)** (non-linear relationships)
 - **Shapley values** (feature importance in model predictions).

Key Takeaway: High-quality, representative data with strong predictive features is essential for effective ML. If data is insufficient, unreliable, or lacks predictive power, ML may not be the right solution.

1.1.2 Framing an ML problem:

1. Define the Ideal Outcome & Model's Goal

- **Ideal Outcome:** What should the product/feature achieve? (From earlier "State the goal" step.)
- **Model's Goal:** How will ML contribute? (E.g., "Predict rain probability" for a weather app.)

2. Identify the Model's Output

- **Predictive ML:** Use for classification (e.g., spam detection) or regression (e.g., sales forecasting).
- **Generative AI:** Use for content creation (e.g., text, images) or natural language tasks (e.g., summarization).
- **Proxy Labels:** If direct labels are missing, use substitutes (e.g., "video likes" as a proxy for "user satisfaction").

3. Generative AI Implementation Options

Since training from scratch is costly, consider:

- **Distillation:** Train a smaller model using synthetic data from a larger one.
- **Fine-tuning:** Adapt a pre-trained model (e.g., GPT, Stable Diffusion) on domain-specific data.
- **Prompt Engineering:** Guide output via instructions/examples (no model retraining).
- **Evaluation:** Use test datasets (e.g., compare AI summaries to human ones).

4. Define Success Metrics

- **Success:** Business impact (e.g., "Users spend 20% more time on the app").
- **Failure:** No improvement (e.g., "No increase in feature usage").
- **Evaluation Metrics:** Model performance (e.g., accuracy, AUC)—but these don't always align with success.
- **Key Question:** Can improving the model move you closer to success?

5. Assess Model Improvement Potential

- **Not good enough, but continue:** Needs work; not production-ready yet.
- **Good enough, and continue:** Works but can be refined.

- **Good enough, but can't improve:** Peak performance; deploy as-is.
- **Not good enough, never will be:** Abandon; no ROI in further training.

6. Measure & Iterate

- **Frequency:** Track success metrics regularly (e.g., weeks/months post-launch).
- **Root-Cause Failure Analysis:** Diagnose why metrics aren't met (e.g., model recommends clickbait, hurting engagement).

Key Takeaways

- Clearly define **what success looks like** (business impact, not just model accuracy).
- Choose **predictive ML or generative AI** based on output needs.
- Use **proxy labels** if direct data is unavailable.
- For generative AI, leverage **pre-trained models + fine-tuning/prompt engineering**.
- Continuously **evaluate if model improvements justify costs**.
- **Failure isn't always the model's fault**—sometimes the problem framing or metrics need adjustment.

This structured approach ensures ML solutions align with real-world goals and deliver measurable value.

1.2 Managing ML Projects:

1.2.1 Development Phases:

Machine Learning Project Lifecycle

1. Ideation & Planning Phase

Goal: Determine if ML is the right solution for the business problem.

Key Tasks:

- Frame the problem in ML terms (predictive vs. generative, supervised vs. unsupervised).
- Assess feasibility (data availability, ROI, constraints).
- Define success metrics aligned with business goals.

Outcome: A design document outlining the ML approach, data needs, and evaluation criteria.

Challenge: Underestimating data exploration and metric definition time.

2. Experimentation Phase

Goal: Develop a viable ML model that solves the problem.

Key Tasks:

- Iterate on features, model architectures, and hyperparameters.
- Validate model performance against benchmarks.

Outcome: A "good enough" model for production.

Challenge:

- New practitioners often overlook the need for robust experiment tracking (e.g., tools like MLflow, Weights & Biases).
 - Hundreds of experiments may be needed to find an optimal solution.
-

3. Pipeline Building & Productionization Phase

Goal: Deploy scalable, maintainable ML systems.

Key Tasks:

- Build automated pipelines for data processing, training, and inference.

- Implement monitoring for data drift, model decay, and system health.
Outcome: Validated pipelines deployed in production.
Challenge:
 - Complexity of production infrastructure (e.g., feature stores, CI/CD for ML).
 - ML-specific monitoring (e.g., prediction skew, concept drift) is often underestimated.
-

4. End-to-End ML Workflow

- **Iterative Process:** Phases often loop back (e.g., production metrics may trigger re-experimentation).
- **Cross-Functional Collaboration:** Requires alignment between data scientists, engineers, and product teams.

Key Takeaways

1. **Start Small:** Prove feasibility in ideation before scaling.
2. **Experimentation is Core:** Allocate time/resources for iterative model development.
3. **Production ≠ Endgame:** 80% of effort often goes into deployment and maintenance.
4. **Monitor Everything:** Data quality, model performance, and infrastructure health are critical post-deployment.

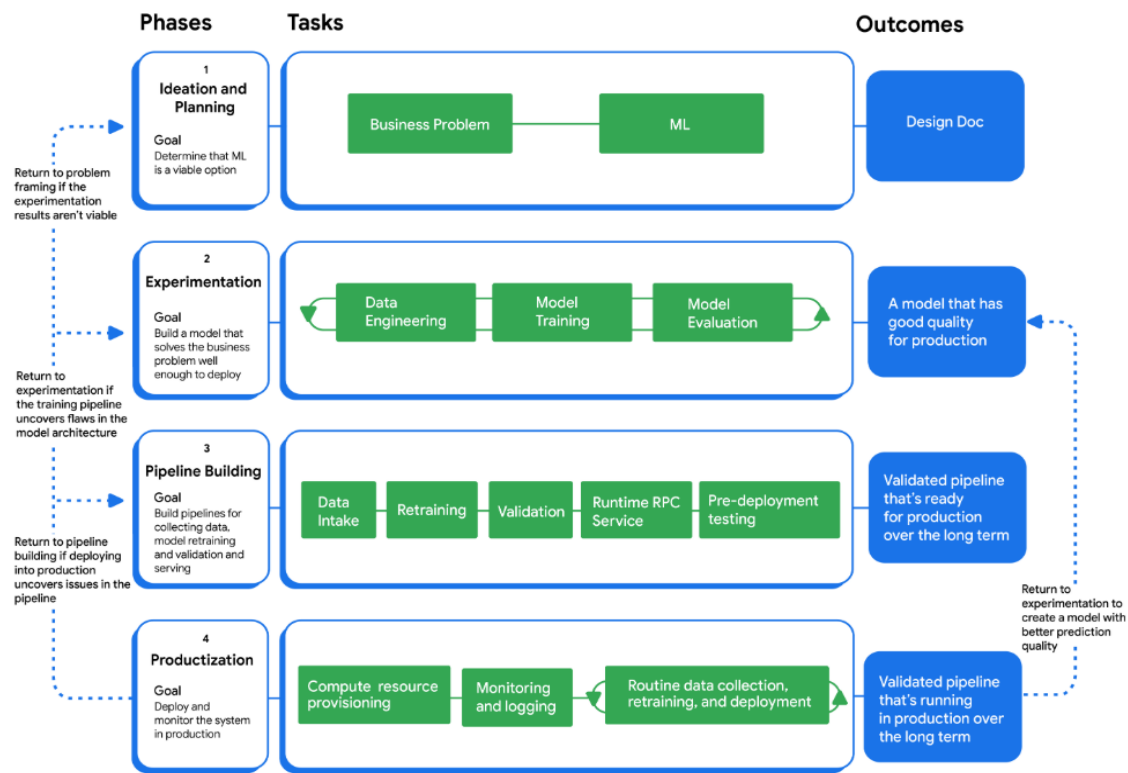
Example Workflow:

- **Google Maps (ETA Prediction):**
 - *Ideation:* "Can we predict travel time more accurately than heuristic rules?"
 - *Experimentation:* Tested 100s of features (traffic, weather, events).
 - *Productionization:* Built pipelines to retrain models hourly with real-time data.

Avoid Pitfalls:

- Underestimating data engineering complexity.
- Treating model deployment as a one-time task.

This phased approach ensures ML projects deliver long-term business value.



1.2.2 Assembling a Team:

Machine Learning Team Composition

Core Roles and Responsibilities

Role	Key Skills	Primary Deliverables
ML Product Manager	Business-ML alignment, requirements definition	Product Requirements Document (PRD)
Engineering Manager	Team leadership, project planning	Design docs, project plans, performance reviews
Data Scientist	Statistical analysis, feature prototyping	Data insights, model prototypes, visualizations

Role	Key Skills	Primary Deliverables
ML Engineer	Model development & productionization	Deployed models meeting quality SLAs
Data Engineer	Data infrastructure & pipelines	Productionized ETL pipelines with monitoring
DevOps Engineer	Model serving infrastructure	Automated deployment & monitoring systems

Note: In smaller teams, individuals often wear multiple hats.

Critical Team Practices

1. Process Documentation

- Essential for aligning cross-functional teams
- Should cover:
 - Data provenance and validation
 - Feature engineering standards
 - Model training/evaluation workflows
 - Production deployment procedures
 - Monitoring and alerting protocols

2. Performance Evaluation Framework

- Measure contributions beyond project success:
 - Code quality and documentation
 - Process improvements
 - Knowledge sharing
 - Experimental rigor
- Recognize valuable "failed" experiments that provide learning

3. Tool Standardization

- Establish consensus on:

- Experiment tracking (MLflow, Weights & Biases)
- Feature stores
- Model serving platforms
- Monitoring solutions

Implementation Considerations

For New Teams:

1. Define development lifecycle phases upfront
2. Document data governance policies early
3. Implement model version control from day one

For Existing Teams:

1. Conduct regular process audits
2. Maintain living documentation
3. Schedule cross-role knowledge sharing sessions

Common Pitfalls to Avoid

1. **Role Ambiguity**
 - Clear RACI matrices prevent overlap/gaps
2. **Documentation Debt**
 - Allocate 20% of sprint time for documentation
3. **Evaluation Myopia**
 - Balance business metrics with engineering quality
4. **Tool Fragmentation**
 - Standardize while allowing innovation sandboxes

Best Practice Examples

Google's Approach:

- "ML Design Docs" template for all projects
- Centralized feature store for all teams
- Standardized model cards for documentation

Startup Adaptation:

- Combined Data Scientist/ML Engineer roles
- Lightweight MLflow tracking instead of complex systems
- Weekly "show your work" sessions for knowledge sharing

This framework ensures teams can maintain velocity while producing production-grade ML systems. The key is balancing structure with flexibility - enough process to prevent chaos, but not so much as to stifle innovation.

1.2.3 Working With Stakeholders:

ML projects have multiple stakeholders with varying levels of involvement and responsibilities. Early involvement and effective collaboration with stakeholders is essential for developing the right solution, managing expectations, and ultimately for a successful ML implementation.

As early as possible, define your project's stakeholders, the expected deliverables, and the preferred communication methods.

Be sure to include them in your list of stakeholders, as well as any other teams who need to approve aspects of your ML solution.

Deliverables

Each stakeholder might expect different deliverables at each phase of the project. Here's a list of common deliverables.

- **Design doc.** Before you write a line of code, you'll most likely create a design doc that explains the problem, the proposed solution, the potential approaches, and possible risks. Typically, the design doc functions as a way to receive feedback and address questions and concerns from the project's stakeholders.
- **Experimental results.** You must communicate the outcomes from the experimentation phase. You'll typically include the following:
 - The record of your experiments with their hyperparameters and metrics.
 - The training stack and saved versions of your model at certain checkpoints.
- **Production-ready implementation.** A full pipeline for training and serving your model is the key deliverable. At this phase, create documentation for future engineers that explain modeling decisions, deployment and monitoring specifics, and data peculiarities.

You should align early with your stakeholders on their expectations for each phase of the project.

Keep in mind

In some cases, stakeholders might not understand the complexities and challenges of ML. This can make getting projects prioritized and executed difficult. For example, some stakeholders might assume that ML is similar to traditional software engineering practices with deterministic outcomes. They might not understand why the project's progress is stalled or why a project's milestones are non-linear.

To manage stakeholder expectations, it's critical to be clear about the complexities, timeframes, and deliverables at each stage of your project.

1.2.4 Ideation and Planning Phase:

i. Feasibility

- **Goal:** Determine if ML is the right solution for the problem.
- **Key Activities:**
 - Assess whether the problem requires ML (vs. rules/heuristics).
 - Evaluate data availability, quality, and relevance.
 - Estimate costs (data labeling, compute resources, engineering effort).
 - Identify potential risks (bias, scalability, regulatory constraints).
- **Outcome:** Go/no-go decision on using ML.

ii. Planning

- **Goal:** Define how the ML solution will be built and deployed.
- **Key Activities:**
 - Draft a **design doc** outlining:
 - Problem statement and success criteria.
 - Model approach (e.g., classification, regression, LLMs).
 - Data sources and feature engineering strategy.
 - Infrastructure needs (cloud, on-prem).
 - Team roles and timelines.
 - Align stakeholders on priorities and trade-offs.
- **Outcome:** Approved project plan with clear milestones.

iii. Measuring Success

- **Goal:** Establish how the model's performance will be evaluated.
- **Key Activities:**
 - Define **business metrics** (e.g., "Increase user engagement by 20%").
 - Choose **model metrics** (accuracy, precision-recall, AUC, etc.).
 - Set up baseline comparisons (e.g., performance vs. existing rules).
- **Outcome:** Agreement on KPIs for each phase (experimentation → production).

1.2.5 Experiments to Production Phase:

i. Experiments

- **Goal:** Develop a working model prototype.
- **Key Activities:**
 - Exploratory Data Analysis (EDA) to identify useful features.
 - Train and compare multiple models (e.g., Random Forest vs. Neural Networks).
 - Optimize hyperparameters (using tools like Optuna or GridSearch).
 - Evaluate against validation/test sets.
- **Outcome:** A model that meets predefined performance thresholds.

ii. ML Pipelines

- **Goal:** Build scalable, automated workflows for model training and inference.
- **Key Activities:**
 - Create **data pipelines** (cleaning, feature engineering, splitting).
 - Set up **training pipelines** (automated retraining, versioning).
 - Design **serving pipelines** (API endpoints, batch predictions).
 - Implement **monitoring** (data drift, model decay).
- **Outcome:** Reliable pipelines that reduce manual effort and errors.

iii. Productionization

- **Goal:** Deploy the model for real-world use.
- **Key Activities:**
 - Integrate with existing systems (APIs, databases, user interfaces).
 - Optimize for performance (latency, throughput) and cost.
 - Set up logging, alerts, and dashboards.
 - Plan for model updates and rollback strategies.
- **Outcome:** A live ML system delivering value to users.

Key Differences Between Phases

Aspect	Ideation/Planning	Experiments → Production
Focus	"Should we do this?"	"How do we do this well at scale?"
Outputs	Design docs, feasibility reports	Trained models, pipelines, deployed systems
Stakeholders	Business leaders, domain experts	Engineers, data scientists, DevOps
Risks	Picking the wrong problem	Technical debt, model failure in production

Best Practices

1. **Iterate Early:** Validate feasibility with small experiments before full-scale development.
2. **Document Everything:** From experiment results to pipeline configurations.
3. **Monitor Rigorously:** Catch model degradation or data shifts post-deployment.
4. **Align Metrics:** Ensure model performance ties to business outcomes (e.g., "Higher AUC → increased revenue").

Example Workflow

Problem: Predict customer churn for a SaaS company.

- **Feasibility:** Confirmed historical churn data exists; ML beats rule-based thresholds.
- **Experiments:** XGBoost outperforms logistic regression (AUC 0.92 vs. 0.85).
- **Productionization:** Deployed as a real-time API used by the customer support team.

This structured approach minimizes wasted effort and maximizes the chances of delivering a successful ML solution.

1.3 Decision Forests:

Decision forests provide the following benefits:

- They are **easier to configure** than neural networks. Decision forests have **fewer hyperparameters**; furthermore, the hyperparameters in decision forests provide **good defaults**.
- They **natively handle** numeric, categorical, and missing features. This means you can write far less preprocessing code than when using a neural network, saving you time and reducing sources for error.
- They often give **good results out of the box**, are robust to noisy data, and have interpretable properties.
- They infer and train on small datasets (< 1M examples) **much faster** than neural networks.

Decision forests produce great results in machine learning competitions, and are heavily used in many industrial tasks.

Decision forests are most effective when you have a tabular dataset (data you might represent in a spreadsheet, csv file, or database table). Tabular data is one of the most common data formats, and decision forests should be your “go-to” solution for modeling it.

Table 1. An example of a tabular dataset.

Number of legs	Number of eyes	Weight (lbs)	Species (label)
2	2	12	Penguin
8	6	0.1	Spider
4	2	44	Dog
...	...	...	...

1. Decision Trees Fundamentals

What Are Decision Trees?

- **Overview:**
Tree-like models that make decisions via recursive splits on feature values.
 - **Nodes:** Decision points (features)
 - **Edges:** Outcomes of decisions (feature thresholds)
 - **Leaves:** Final predictions (class labels/regression values)
 - **Conditions:**
Splits are chosen to maximize information gain (e.g., Gini impurity/entropy for classification, variance reduction for regression).
 - **Check Your Understanding:**
Q: *What metric would you use to split a node for a classification task?*
A: Gini impurity or entropy.
-

2. Training Decision Trees

Growing Decision Trees

- **Process:**
 1. Start at root with all data.
 2. Split data recursively using the "best" feature/threshold.
 3. Stop when a stopping criterion is met (max depth, min samples per leaf).
- **Binary Classification:**
Trees classify data into one of two classes (e.g., spam/not spam).
- **Overfitting & Pruning:**
 - **Problem:** Deep trees memorize training data → poor generalization.
 - **Solution:** Prune by removing nodes that add little predictive power.
- **Variable Importance:**
Measures how much a feature reduces impurity across the tree.
- **Check Your Understanding:**
Q: *Why prune a decision tree?*
A: To reduce overfitting and improve generalization to unseen data.

3. Decision Forests & Random Forests

What Are Decision Forests?

- Ensembles of multiple decision trees to improve accuracy and robustness.

Random Forests

- **Key Idea:**
 - Train many trees on random subsets of data (bootstrap samples).
 - At each split, consider only a random subset of features.
- **Advantages:**
 - Reduces overfitting via averaging (bagging).
 - Handles noisy data better than single trees.
- **Check Your Understanding:**

Q: *How do random forests ensure diversity among trees?*

A: By using random subsets of data *and* features for each tree.

4. Gradient Boosted Decision Trees (GBDT)

Introduction

- **Goal:** Sequentially build trees to correct errors of previous trees.
- **Key Difference vs. Random Forests:**
 - Trees are built **additively** (each tree learns from prior mistakes).
 - Uses gradient descent to minimize loss (e.g., log loss, MSE).

Overfitting & Regularization

- **Techniques:**
 - **Learning rate:** Shrinks each tree's contribution.
 - **Early stopping:** Halts training if validation performance plateaus.
 - **Subsampling:** Uses random data subsets per iteration (stochastic boosting).
-

5. Advanced Topics

- **Out-of-Bag (OOB) Evaluation:**
 - In random forests, each tree is trained on ~63% of data; the remaining 37% ("out-of-bag") can estimate model performance without a separate validation set.
 - **Other Topics:**
 - **Feature engineering** for trees (e.g., binning continuous variables).
 - **Interpretability** (SHAP values, tree visualization).
-

Example Workflow: Spam Detection

1. **Single Tree:**
 - Splits on keywords ("free", "win") → 85% accuracy.
 2. **Random Forest:**
 - Combines 100 trees → 92% accuracy (reduces false positives).
 3. **GBDT (XGBoost):**
 - Iteratively improves → 95% accuracy by focusing on misclassified emails.
-

Key Takeaways

- **Decision Trees:** Simple but prone to overfitting.
- **Random Forests:** Robust via diversity (bagging + random features).
- **GBDT:** High accuracy via iterative error correction (boosting).

Further Checks:

- Try building a decision tree on a small dataset (e.g., Iris or Titanic).
- Compare Random Forest vs. GBDT performance on Kaggle.

This framework ensures a solid grasp of tree-based methods, from basics to advanced ensembles.

1.4 Recommendation Systems:

1. Background

Large-Scale Recommendation Systems

- **Purpose:** Filter vast item catalogs (e.g., videos, products) to personalize user experiences.
- **Scale Challenges:** Millions of users/items require efficient algorithms (e.g., approximate nearest neighbors).

Terminology

- **User-Item Matrix:** Tracks interactions (e.g., clicks, ratings).
- **Cold Start:** Problem of recommending to new users/items with no history.
- **Engagement Metrics:** CTR (click-through rate), watch time, conversions.

Check Your Understanding

Q: *Why is scalability critical in recommendation systems?*

A: To handle real-time predictions across massive datasets (e.g., YouTube's 500+ hours of uploads/minute).

2. Candidate Generation

Overview

- **Goal:** Narrow millions of items to hundreds of plausible candidates.
- **Methods:**
 - **Content-Based Filtering:** Uses item/user features.
 - **Collaborative Filtering:** Leverages user-item interactions.

Content-Based Filtering

- **Basics:** Recommends items similar to those a user liked before (e.g., "movies with same genre").
 - **Features:** Item metadata (title, tags), user preferences.
 - **Similarity Metrics:** Cosine similarity, TF-IDF.

- **Pros/Cons:**
 - ✓ Handles cold start (if item features exist).
 - ✗ Limited serendipity (no cross-user insights).

Collaborative Filtering (CF)

- **Basics:** Recommends items based on similar users' preferences.
 - **User-Item Matrix:** Sparse matrix of interactions (e.g., ratings).
- **Matrix Factorization:**
 - Decomposes the matrix into latent factors (e.g., "comedy" vs. "action" affinity).
 - Algorithms: SVD, ALS (Alternating Least Squares).
- **Pros/Cons:**
 - ✓ Discovers hidden patterns (e.g., niche genres).
 - ✗ Fails on cold start; sensitive to sparsity.

3. Deep Neural Networks (DNNs)

Softmax Model

- **Architecture:**
 - Treats recommendation as classification: Predict the probability a user interacts with item i among N items.
 - Output layer: Softmax over all items (computationally expensive → requires tricks like sampled softmax).
- **Training:**
 - Minimize cross-entropy loss using user/item embeddings.
 - Example: YouTube's DNN candidate generator.

Check Your Understanding

Q: *Why is softmax impractical for large item catalogs?*

A: Computing probabilities over millions of items is resource-intensive (sampling or two-tower models address this).

4. Retrieval, Scoring, & Re-Ranking

Retrieval

- **Goal:** Fast but coarse selection (e.g., approximate nearest neighbors).
- **Tools:** FAISS, ScaNN for vector similarity search.

Scoring

- **Goal:** Rank candidates by predicted engagement (e.g., watch time, CTR).
- **Models:**
 - Logistic regression (interpretable).
 - DNNs (capture complex interactions).

Re-Ranking

- **Goal:** Refine rankings using business rules/DL.
- **Techniques:**
 - **Diversity:** Avoid over-recommending similar items.
 - **Fairness:** Ensure coverage for niche categories.
 - **Context:** Adjust for time/location (e.g., holiday-themed ads).

Example: Netflix Recommendation Pipeline

1. **Candidate Generation:**
 - Collaborative filtering → 500 movies similar to user's past watches.
 2. **Scoring:**
 - DNN predicts likelihood of watching each movie.
 3. **Re-Ranking:**
 - Boosts new releases, balances genres.
-

Key Trade-Offs

Method	Strengths	Weaknesses
Content-Based	Cold-start capable	Limited personalization
Collaborative Filtering	Discovers cross-user patterns	Fails on cold start
DNNs	Handles complex signals	Computationally expensive

Advanced Topics

- **Two-Tower Models:** Separate user/item encoders for efficient retrieval (e.g., TensorFlow Recommenders).
- **Reinforcement Learning:** Optimizes long-term engagement (e.g., TikTok’s reward model).

This framework balances theory and practicality, covering everything from foundational algorithms to cutting-edge DNN approaches.

New chat

1.5 Long-Short-Term-Memory (LSTMs):

1. Why LSTMs?

Before LSTMs, **RNNs (Recurrent Neural Networks)** were the main way to deal with sequence data like text, speech, or time series.

RNNs work by feeding the output from one time step back into the network for the next time step.

However, **vanilla RNNs** suffer from:

- **Vanishing gradients** – hard to learn dependencies far back in the sequence.
- **Exploding gradients** – weights can blow up when backpropagating through many steps.
- They forget long-term dependencies — struggle to connect information far apart.

LSTMs were introduced by Hochreiter & Schmidhuber (1997) to solve this.

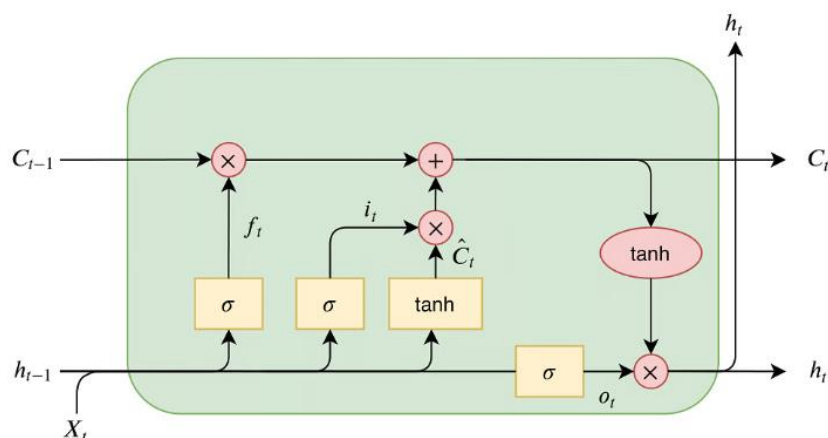
2. Key Idea of LSTMs

LSTMs add a **cell state** — like a conveyor belt — that runs through the sequence with minimal changes.

They use **gates** (sigmoid-activated layers) to control:

1. What to remember.
2. What to forget.
3. What to output.

This allows them to keep information over long time spans without gradients vanishing.



3. The LSTM Structure

At each time step t , LSTM has:

- **Input vector** x_t (current data point, e.g., a word embedding).
- **Hidden state** h_{t-1} (short-term memory).
- **Cell state** C_{t-1} (long-term memory).

It produces:

- New hidden state h_t .
 - New cell state C_t .
-

3.1 Gates and Their Roles

Each gate is a small neural network layer that uses **sigmoid** activation (σ), which outputs values in $[0,1]$ representing “how much” to allow through.

(1) Forget Gate

Decides what old information to drop from the cell state.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

- f_t close to 0 $\rightarrow$ forget that part of C_{t-1} .
 - f_t close to 1 $\rightarrow$ keep it.
-

(2) Input Gate

Decides what new information to add to the cell state.

Two steps:

- **Candidate values** (new content we might add):

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

- **Gate to decide how much to add:**

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

(3) Cell State Update

Combine old and new information:

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

- First term $\rightarrow$ old state scaled by forget gate.
 - Second term $\rightarrow$ new candidate scaled by input gate.
-

(4) Output Gate

Decides what the hidden state should be.

- Gate:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

- Hidden state:

$$h_t = o_t * \tanh(W_c \cdot C_t) \quad h_t = o_t * \tanh(C_t)$$

3.2 Data Flow Summary

At each time step:

1. **Forget** old unnecessary info.
2. **Add** new candidate info.
3. **Update** cell state.
4. **Output** the current hidden state.

The cell state acts as **long-term memory**, and the hidden state acts as **short-term working memory**.

4. Why This Works

- **Gradient flow:** The cell state updates mostly by addition and scaling (not multiplication through time), so gradients don't vanish easily.
 - **Selective memory:** The gates learn what to remember and what to discard.
 - **Long context handling:** LSTMs can capture dependencies over hundreds of time steps.
-

5. Applications

- **NLP:** Machine translation, sentiment analysis, text generation.
 - **Speech recognition:** Handling long audio sequences.
 - **Time-series forecasting:** Stock prices, weather predictions.
 - **Video analysis:** Frame-by-frame context retention.
-

6. Drawbacks

- More complex and slower than vanilla RNNs (4 weight matrices per gate).
 - Sometimes overkill for shorter sequences.
 - Nowadays, **GRUs** or even **Transformers** are often preferred for efficiency.
-

1.6 Gated Recurrent Units (GRUs):

1. Why GRUs?

LSTMs are powerful but:

- Have 3 gates and two states (C_t cell state + h_t hidden state).
- Are slower and have more parameters.

GRUs (Cho et al., 2014) simplify this by:

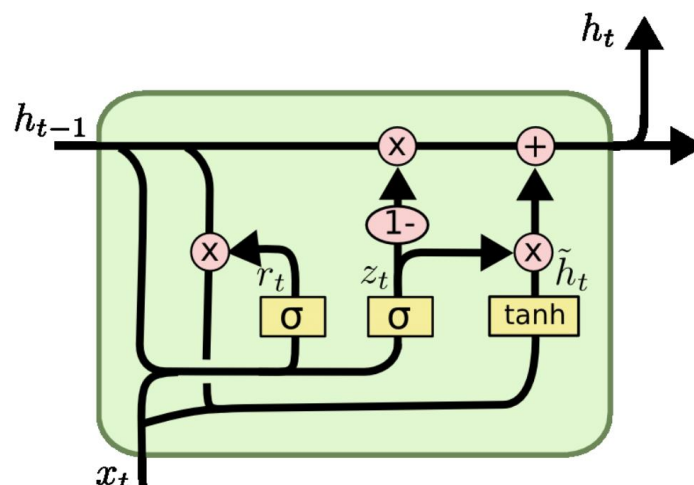
- Using only **two gates**.
- Combining the cell state and hidden state into a **single vector** (h_t).
- Often training faster with similar accuracy.

2. Components

At time step t :

- $x_t \rightarrow$ input vector at time t
- $h_{t-1} \rightarrow$ previous hidden state (also acts as memory)
- Output: $h_t \rightarrow$ current hidden state

3. GRU Gates and Operations



(1) Update Gate

Controls how much of the previous hidden state is kept vs. replaced with new information.

$$z_t = \text{sigmoid}(W_z * [h_{(t-1)}, x_t] + b_z)$$

- z_t close to 1 $\rightarrow$ keep more of the old state.
 - z_t close to 0 $\rightarrow$ update with new info.
-

(2) Reset Gate

Controls how much of the previous hidden state to forget before calculating the candidate state.

$$r_t = \text{sigmoid}(W_r * [h_{(t-1)}, x_t] + b_r)$$

- r_t close to 0 $\rightarrow$ ignore past memory.
 - r_t close to 1 $\rightarrow$ use full past memory.
-

(3) Candidate Hidden State

New potential hidden state based on current input and a reset version of past hidden state.

$$\tilde{h}_t = \tanh(W_h * [r_t * h_{(t-1)}, x_t] + b_h)$$

(4) Final Hidden State Update

Blend between old hidden state and new candidate based on the update gate.

$$h_t = (1 - z_t) * \tilde{h}_t + z_t * h_{(t-1)}$$

4. Step-by-Step Flow

1. **Update gate** (z_t) decides how much old info to keep.
 2. **Reset gate** (r_t) decides how much past info to forget.
 3. Compute **candidate state** ($\tilde{h}_t$) using possibly reset memory.
 4. Mix old state and candidate to get new hidden state (h_t).
-

5. Key Differences from LSTM

- No separate cell state C_t .
 - Only 2 gates (update, reset) vs 3 gates (forget, input, output) in LSTM.
 - Fewer parameters → faster training.
 - Often works as well as LSTM on many tasks.
-

6. Common Applications

- Machine translation
 - Text classification
 - Speech recognition
 - Time series prediction
-

1.7 Encoder Decoder Architecture:

1. What is Encoder–Decoder?

The **Encoder–Decoder architecture** is a neural network design used for tasks where:

- Input and output are both sequences (possibly of different lengths).
- Examples:
 - Machine Translation (English → French)
 - Text Summarization
 - Image Captioning
 - Speech-to-Text

It consists of **two main parts**:

- **Encoder** → processes the input sequence into a fixed-size context representation.
 - **Decoder** → generates the output sequence step-by-step from the context.
-

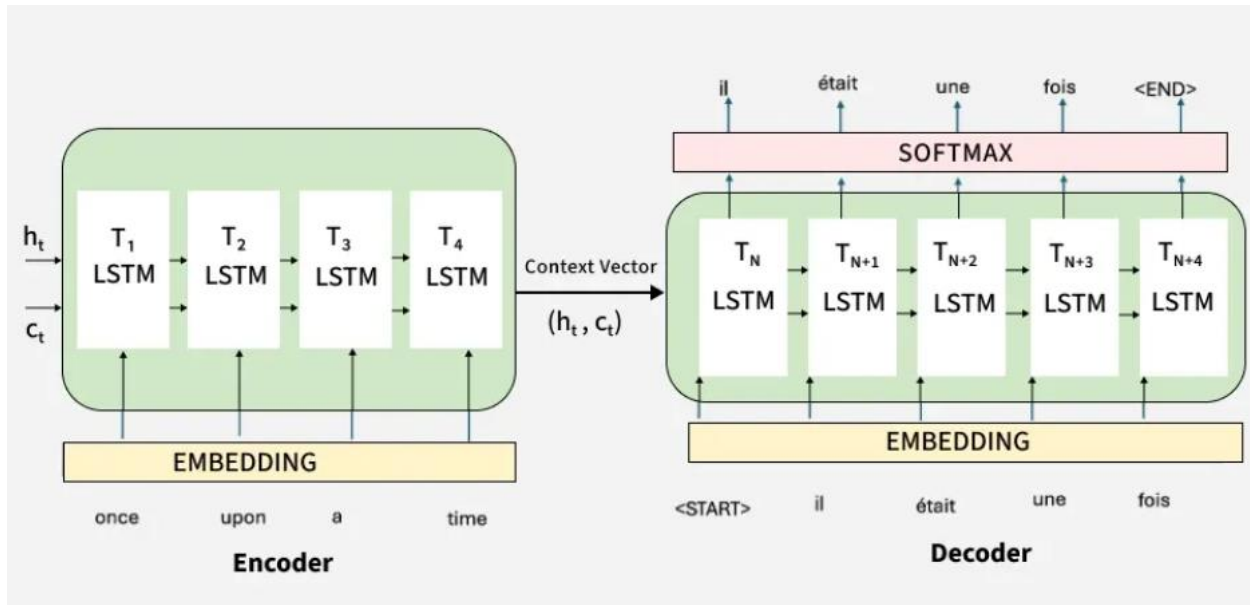
2. General Workflow

1. Encoder

- Reads the input sequence one element at a time.
- Produces hidden states at each step.
- Final hidden state (or a processed version of all states) becomes the **context vector** — a compressed summary of the input.

2. Decoder

- Takes the context vector and produces output one step at a time.
 - Uses its own previous outputs as inputs for the next step (auto-regressive decoding).
 - Can use attention to focus on specific parts of the encoder's output.
-



3. Encoder Equations (using RNN/LSTM/GRU)

Let the input sequence be:

$x_1, x_2, \dots, x_{T_x}$ where T_x is the input length.

For an RNN-style encoder:

$$h_t = \text{RNN_Cell}(x_t, h_{(t-1)})$$

- h_t = hidden state at time t
- Final context vector:

$$C = h_{T_x}$$

(In attention-based models, C can be the set of all hidden states $\{h_1, h_2, \dots, h_{T_x}\}$.)

4. Decoder Equations

Let the output sequence be:

$y_1, y_2, \dots, y_{T_y}$ where T_y is the output length.

The decoder generates:

$$s_t = \text{RNN_Cell}(y_{(t-1)}, s_{(t-1)}, C)$$

- s_t = decoder hidden state at time t
- y_t predicted from:

$$y_t = \text{softmax}(W_o * s_t + b_o)$$

5. Training vs. Inference

- **Training (Teacher Forcing):**
At each step, the decoder receives the correct previous output ($y_{(t-1)}$) from the dataset.
- **Inference (Prediction):**
At each step, the decoder uses its own previous prediction instead.

6. Attention Mechanism (Optional but Common)

Instead of using a single context vector C , the decoder:

- Computes attention weights over all encoder hidden states.
- Produces a weighted sum (dynamic context) for each decoding step:

$\text{score}_{ti} = \text{alignment}(s_{(t-1)}, h_i)$

$\alpha_{ti} = \text{softmax}(\text{score}_{ti})$

$C_t = \sum_i (\alpha_{ti} * h_i)$

This helps the decoder focus on relevant parts of the input for each output step.

7. Applications

- **Machine Translation** (seq-to-seq)
- **Speech Recognition** (audio $\rightarrow$ text)
- **Text Summarization** (long text $\rightarrow$ short summary)
- **Image Captioning** (image features $\rightarrow$ text)

1. The Core Difference

- **LSTMs/GRUs:**
These are *sequence-processing cells*. They take in sequences and produce hidden states. They can process one sequence at a time (input $\rightarrow$ output at each step).

- **Encoder–Decoder:**

This is a *framework* that uses one network (often an LSTM/GRU) to **encode** the input sequence into a representation, and another network to **decode** that representation into an output sequence.

2. Why LSTMs/GRUs Alone Aren't Enough

Imagine **machine translation** (English → French):

If you use a single LSTM:

- It reads a word in English, outputs a word in French at the same time.
- Problem: Translation often requires looking ahead or reordering words.
- Input and output lengths may be different.
- It can't easily “pause” output until it understands the whole input.

With Encoder–Decoder:

- The **Encoder** LSTM reads the entire English sentence first, builds a context vector summarizing it.
 - The **Decoder** LSTM starts generating French words based on that context, possibly reordering and changing grammar.
 - This separation allows:
 - Different input/output lengths.
 - Global understanding before output.
 - Better handling of dependencies.
-

3. Why This Matters

Tasks like:

- Translation
- Summarization
- Question Answering
- Image Captioning

... require that **you first understand the whole input before generating the output** — not just process input and output in lockstep.

4. How They Work Together

- Encoder–Decoder is like a **two-stage machine**:
 1. **Encoder** → Comprehend the input sequence.
 2. **Decoder** → Generate the output sequence.
 - LSTMs or GRUs are like the **engine parts** inside each stage.
You can replace them with:
 - LSTMs
 - GRUs
 - Transformers
 - CNNs (for some types of encoding)
 - etc.
-

5. Quick Analogy

Think of it like:

- **LSTM/GRU** = a type of motor that processes sequences.
- **Encoder–Decoder** = a car design that uses one motor to process input and another to generate output.

You can't drive with just a motor — you need a whole system.

2. Project Work:

2.1 Text Generation (Character Level Prediction):

- **Objective:** Train an LSTM model to learn character sequences from the Tiny Shakespeare dataset and generate new text in Shakespeare's style.
-

Steps Involved:

1. Data Loading and Cleaning:

- Read raw text from tiny shakespeare.txt.
- Converted all characters to lowercase.
- Replaced newlines/tabs with spaces and removed multiple spaces.
- Produced a clean, continuous text string suitable for character-level modeling.

2. Character Mapping:

- Extracted all unique characters from the cleaned text.
- Created two mappings:
 - **char_to_idx:** Maps each character to a unique integer index.
 - **idx_to_char:** Maps each index back to its character.
- Converted the text into sequences of integer indices for model input (inputs) and shifted sequences for targets (targets).

3. Sequence Preparation:

- Defined a sequence length of 100 characters.
- Created overlapping input–target pairs where each input sequence predicts the next sequence of characters.
- Final arrays: x_seq (input sequences) and y_seq (corresponding target sequences).

4. Model Architecture:

- **Embedding Layer:** Converts integer character indices into dense vector representations (dimension = 64).

- **LSTM Layer:** 256 units, return_sequences=True to output predictions for every timestep in the sequence.
- **Dense Output Layer:** Fully connected layer with softmax activation to output probability distribution over the vocabulary at each timestep.

5. **Model Compilation and Training:**

- Loss Function: sparse_categorical_crossentropy (suitable for integer targets).
- Optimizer: Adam optimizer for faster convergence.
- Metrics: Accuracy measured at the character level.
- Trained on the first 50,000 sequences, batch size = 256, for 50 epochs.

6. **Training Monitoring:**

- Plotted **training loss** and **accuracy** over epochs.
- Dual-axis plot used to visualize the inverse relationship between loss and accuracy during training.

7. **Text Generation (Greedy Search):**

- Function reads a starting text prompt.
- Predicts the next character by selecting the index with the highest probability from the last timestep (np.argmax).
- Slides the input window forward by 1 character after each prediction.
- Continues until the desired length is reached.

8. **Text Generation (Temperature Sampling):**

- Introduced a temperature parameter to control randomness.
- Higher temperature → more diverse but riskier predictions.
- Lower temperature → more predictable and conservative text.
- Used probabilistic sampling from the adjusted probability distribution instead of always picking the most likely character.

9. **Model Saving and Loading:**

- Saved the trained model in .keras format.
- Saved char_to_idx and idx_to_char mappings using pickle.

- Demonstrated loading both the model and mappings for future inference without retraining.
-

2.2 Capstone Project – Next Word Prediction:

- **Objective:** Train an LSTM-based language model to predict the next word in a sequence.
- **Data Preprocessing:**
 - Used Tokenizer to convert words to numerical indices (word_index length = 700).
 - Generated **input sequences** by progressively increasing tokens in each line.
 - Determined **max sequence length** for padding.
 - Split into features X (all but last token) and labels y (last token).
 - One-hot encoded y with to_categorical() for 700 vocabulary classes.
- **Model Architecture:**
 - Embedding(700, 100) to convert word indices into dense vectors.
 - Two LSTM layers (256 units with return sequences, then 128 units).
 - Dense output layer with softmax for 700-word vocabulary.
 - Loss = categorical_crossentropy, Optimizer = adam, Metric = accuracy.
- **Training:**
 - Trained for **150 epochs** on prepared sequences.
- **Inference:**
 - Starts from seed text "General Task Flow".
 - Iteratively:
 - Tokenize & pad to max length.
 - Predict next word (argmax on softmax output).
 - Append predicted word to text.
 - Repeat for 20 words with 2-second delay between predictions.
- **Saving:**

- Saved model (next_word_generator.keras) **without optimizer state**.
 - Saved tokenizer (tokenizer.pkl) for consistent preprocessing at inference.
-

2.3 Translation Task using encoder-decoder architecture:

Translation Task with Seq2Seq LSTM

Objective

Build a **sequence-to-sequence (Seq2Seq) model** to translate English sentences to Urdu using LSTM-based encoder-decoder architecture.

Key Steps

1. Data Preprocessing

- Tokenized English & Urdu texts (added startseq/endseq for Urdu).
- Padded sequences to fixed lengths.

2. Model Architecture

- **Encoder:** Embedding → LSTM (256 units) → Final states.
- **Decoder:** LSTM (256 units) initialized with encoder states → Dense (softmax).

3. Training

- **Loss:** sparse_categorical_crossentropy.
- **Optimizer:** adam.
- **Epochs:** 20.

4. Inference

- Encoder encodes input → Decoder generates Urdu tokens iteratively.
- Stops at endseq or max length.

Example

Input: "Is everything OK?"

Output: "ہے؟ ٹھیک سب کیا"

Comparison to Next-Word Prediction:

- **Translation:** Seq2Seq with encoder-decoder.
- **Next-Word:** Single LSTM predicting next token in a sequence.

3. Challenges and Approaches:

1. Character Level Prediction:

Challenge: While working with this task the main problem was that when we split the data into characters the size become very large for training and it was taking a lot of time.

Approach: The approach I used was to take a specific amount of data sample for the overall data then train and evaluate the model and it worked.

2. Next Word Prediction:

Challenge: The challenge with next word prediction was the same as with the next word prediction + I tried multiple models along with encoder-decoder architecture but all were overfitting.

Approach: The approach I used is that I fetched a small dummy data and trained the model and but still there was a lot of overfitting.

LSTMs and GRUs struggle with capturing patterns in long sequential data will try to use attention mechanism and LLMs in future to tackle this problem.

3. Translation using encoder-decoder:

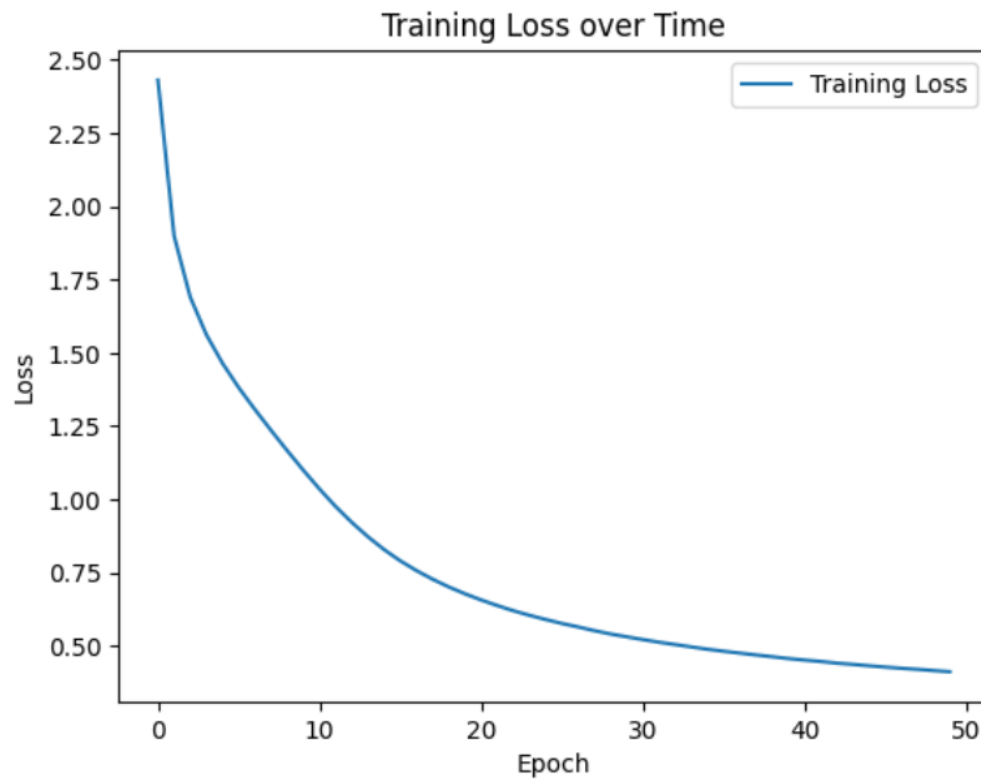
Challenge: The problem I faced was that I used English to urdu dataset but the data was very small and the model was overfitting.

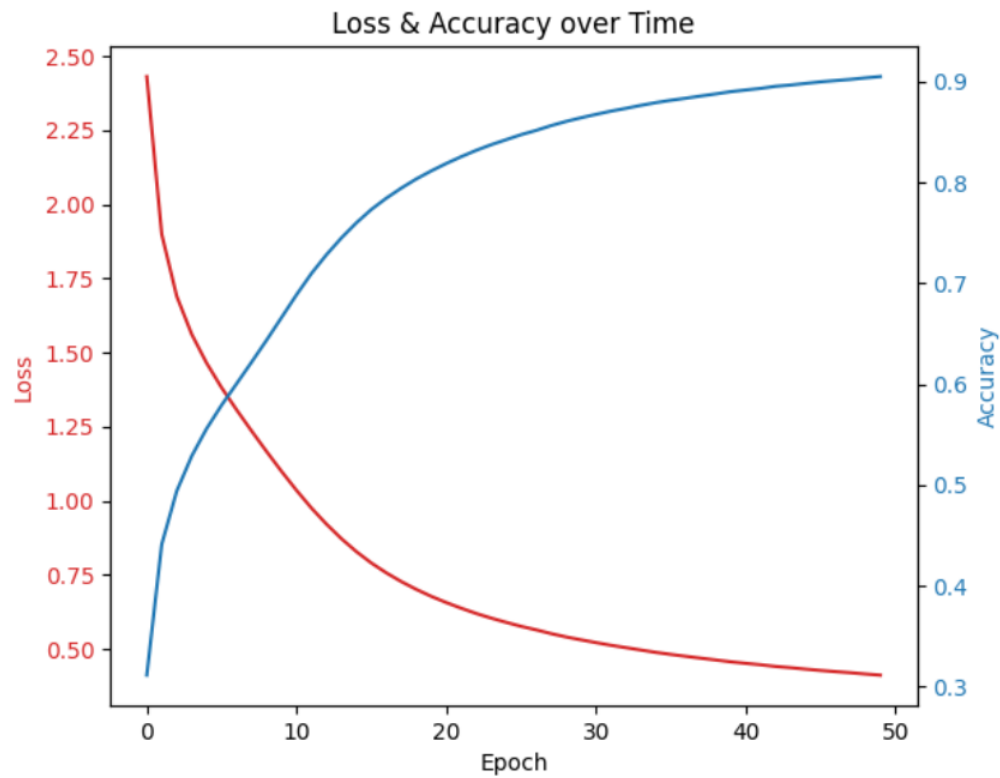
Approach: Adopted multiple approaches but still no improvement will try to find a larger and diverse dataset.

4. Result and Analysis:

1. Character Level Prediction:

accuracy: 0.9644 - **loss:** 0.1186





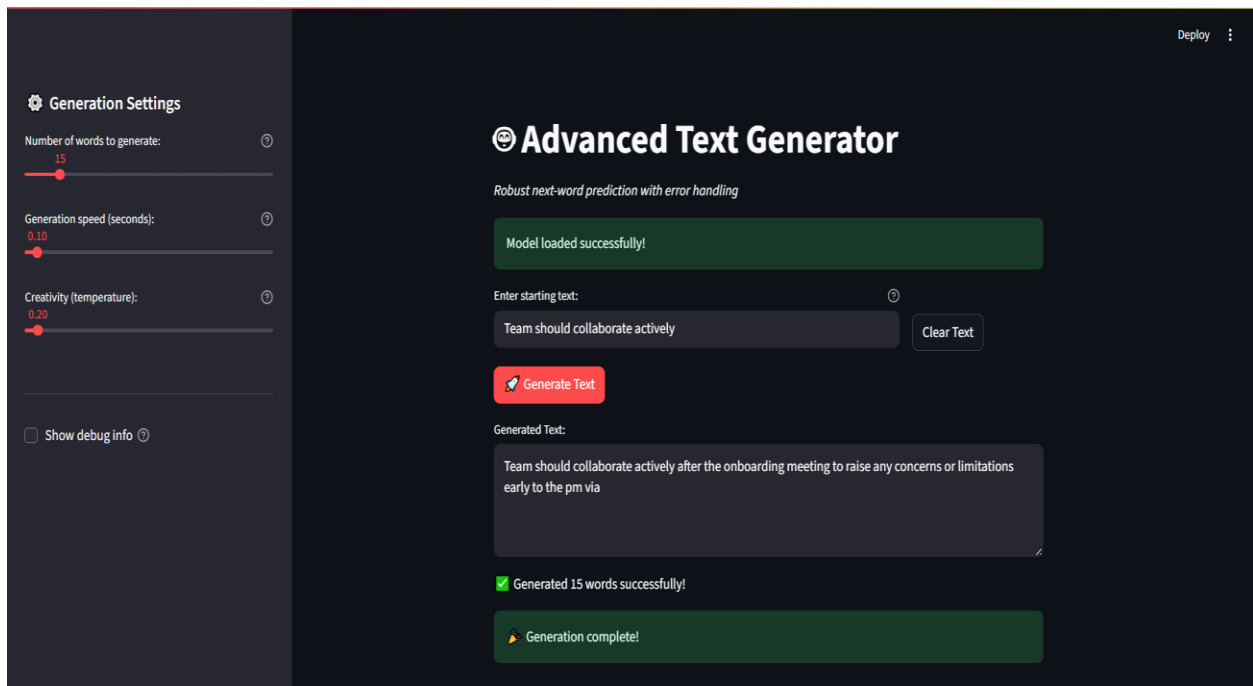
2. Next Word Prediction:

accuracy: 0.9098 - **loss:** 0.2940

input : "General Task Flow"

output: "General Task Flow of the project instead ensure that all delivery delays proper professional within card comments to ensure prompt impact on performance"

Streamlit Application



3. Translation using encoder-decoder:

accuracy: 0.7401 - loss: 1.3754 - val_accuracy: 0.4901 - val_loss: 3.8339

There are very less plots and metrics for this week because the results were not so promising

5. Conclusion:

Through this week, I explored the architecture and mathematics behind **LSTMs, GRUs, and encoder-decoder models** while working on **character-level prediction, next-word prediction, and machine translation tasks**. Key challenges included **data scalability, overfitting, and model limitations in capturing long-term dependencies**.

Future Approaches

1. Attention Mechanism & Transformers

- Replace LSTMs/GRUs with **Transformer-based models** (e.g., BERT, GPT) to better handle long sequences.
- Implement **self-attention** for improved context retention in translation tasks.

2. Larger & Diverse Datasets

- Curate a **balanced, high-quality English-Urdu parallel corpus** to reduce overfitting.
- Use **data augmentation** (e.g., back-translation) for low-resource language pairs.

3. Pre-trained Language Models (LLMs)

- Fine-tune **mT5 or NLLB** (Facebook's multilingual model) for translation.
- Leverage **GPT-like models** for next-word prediction with few-shot learning.

4. Regularization & Optimization

- Experiment with **dropout, weight decay, and gradient clipping** to combat overfitting.
- Use **mixed-precision training** to speed up LSTM/GRU workflows.

5. Hybrid Models

- Combine **CNNs for character-level features + Transformers for word-level context** in NLP tasks.

Goal: Enhance accuracy, reduce training time, and generalize better across tasks. 🚀

6. References:

- <https://www.youtube.com/>
- <https://www.youtube.com/@campusx-official>
- <https://www.kaggle.com/datasets>
- <https://huggingface.co/datasets>
- <http://geeksforgeeks.org/deep-learning/deep-learning-introduction-to-long-short-term-memory/>
- <https://www.geeksforgeeks.org/machine-learning/gated-recurrent-unit-networks/>
- <https://www.geeksforgeeks.org/nlp/encoder-decoder-models/>